

CS-4063 NLP Assignment 3 Report

Transformer-based Review Understanding with RAG-Enhanced Explanation Generation

Name: Shazil | Roll No: 22i-0452 | Course: CS-4063 Natural Language Processing

1. Project Overview

This project implements a complete three-stage NLP pipeline for Amazon review understanding and explanation generation.

- Encoder-only Transformer: predicts sentiment, predicts review length class, and produces fixed-size review embeddings.
- Retrieval module: stores training embeddings and retrieves top-k similar reviews using cosine similarity.
- Decoder-only Transformer: generates short explanations using review text, predicted labels, and retrieved context.
- Forbidden modules were not used: nn.Transformer, nn.MultiheadAttention, and nn.TransformerEncoder.
- Manual components: scaled dot-product attention, multi-head attention, encoder/decoder blocks, residual connections, layer normalization, padding mask, causal mask, and autoregressive generation.

2. Dataset and Preprocessing

Amazon review data was used from three categories: sports, cellphones, and home. A smaller subset was used due to hardware limitations.

Item	Value
Total cleaned samples	5998
Train	4198
Validation	900
Test	900
Sports	2000
Cellphones	1999
Home	1999

- Cleaning steps: lowercase conversion, HTML removal, URL removal, unwanted symbol removal, whitespace normalization, invalid review removal, and duplicate removal.
- Sentiment mapping: 1-2 stars = negative, 3 stars = neutral, 4-5 stars = positive.

- Final sentiment distribution: positive = 4986, negative = 578, neutral = 434.
- Observation: dataset is strongly imbalanced toward positive reviews, which affected sentiment macro F1.

Sentiment	Count
positive	4986
negative	578
neutral	434

3. Derived Feature and Input Representation

The second task for multi-task learning was review_length_class, calculated from token count percentiles.

Length Class	Count
long	2039
short	2032
medium	1927

- Reason: review length is predictable from text and represents brief, moderate, or detailed feedback.
- Vocabulary was built only from training data to avoid data leakage.
- Special tokens used: <PAD>, <UNK>, <BOS>, <EOS>.
- Reviews were tokenized, converted to IDs, padded/truncated, and converted into attention masks.

Input Setting	Value
Vocabulary size	8122
Encoder max sequence length	128
Batch size	32
input_ids shape	[32, 128]
attention_mask shape	[32, 128]

4. Encoder-only Transformer

The encoder-only Transformer was implemented from scratch and used for multi-task review understanding.

Hyperparameter	Value
Vocabulary size	8122
Max length	128
Embedding dimension	128

Hyperparameter	Value
Attention heads	4
Encoder layers	2
Feed-forward dimension	256
Dropout	0.1
Classes	3 sentiment, 3 length
• Architecture: token embedding, sinusoidal positional encoding, multi-head self-attention, feed-forward network, residual connections, layer normalization, dropout, and masked mean pooling. • Outputs: sentiment logits [32, 3], length logits [32, 3], review embedding [32, 128]. • Attention check: 2 attention layers with attention shape [32, 4, 128, 128]. • The 128-dimensional review embedding was saved for retrieval.	

5. Encoder Training and Evaluation

The encoder was trained using a combined multi-task objective:

Total Loss = Sentiment Loss + 0.5 x Length Feature Loss

Training Setting	Value
Epochs	3
Batch size	32
Learning rate	0.0005
Optimizer	Adam
Device	CPU
Loss	Class-weighted cross entropy

Metric	Value
Test Loss	1.0756
Sentiment Accuracy	0.4700
Sentiment Macro F1	0.2971
Length Accuracy	0.9633
Length Macro F1	0.9624
Train embedding shape	(4198, 128)
• Length classification performed very well because token count directly supports this target. • Sentiment classification was harder due to severe class imbalance and the smaller training subset. • Macro F1 is lower because negative and neutral classes had fewer examples.	

- Saved outputs: encoder_trained.pt, encoder_metrics.json, learning_curves.png, train_embeddings.npy, and train_metadata.csv.

6. Retrieval Module

The retrieval module used saved encoder embeddings as an index. Each test review was encoded and compared with all training embeddings.

Retrieval Item	Value
Similarity metric	Cosine similarity
Top-k	3
Train embeddings	(4198, 128)
Test embeddings	(900, 128)
Similarity matrix	(900, 4198)
Top-1 same sentiment rate	0.7356
Average top-1 similarity	0.9815
• Cosine similarity was selected because it compares vector direction and is common for text embeddings. • $k = 3$ was selected to provide useful context while keeping decoder input short. • Retrieval quality was reasonable: many query reviews retrieved same-sentiment examples. • Limitation: retrieval is based on embedding similarity, so wrong encoder predictions can still affect downstream generation. • Saved outputs: retrieval_examples.json, retrieval_analysis.csv, and phase5_summary.json.	

7. Decoder-only Transformer and RAG Generation

The decoder was implemented from scratch and trained to generate explanations from structured prompts.

Decoder Hyperparameter	Value
Decoder vocabulary	8125
Max length	160
Embedding dimension	128
Attention heads	4
Decoder layers	2
Feed-forward dimension	256
Dropout	0.1
Batch size	16

Decoder Hyperparameter	Value
Epochs	2
Learning rate	0.0007
• Prompt format: review text + predicted sentiment + predicted length feature + retrieved reviews + explanation prefix. • Causal mask was used so each token could attend only to previous tokens and itself. • Generation was autoregressive: explanation tokens were generated one by one. • Prompt tokens were ignored in the loss using -100, so loss focused on explanation tokens.	

Decoder Data	Samples
Train decoder samples	1200
Validation samples	200
Test evaluation samples	100

Epoch	Train Loss	Val Loss	Val Perplexity
1	3.0424	0.6061	1.8332
2	0.4178	0.2609	1.2981

System	Loss	Perplexity
Full RAG	2.0449	7.7282
No Retrieval Baseline	4.1092	60.8991
• Full RAG achieved much lower perplexity than the no-retrieval baseline. • This shows that retrieved context helped the decoder predict explanation tokens better. • Saved outputs: decoder_trained.pt, decoder_vocab.json, perplexity.json, generated_examples.json, rag_ablation.csv, and phase6_summary.json.		

8. Generated Examples and Quality Analysis

Example	Prediction	Generated Output / Analysis
1	positive / medium	Generated positive satisfaction explanation. Coherent but repetitive and mentioned short review incorrectly.
2	positive / short	Generated: this review is positive because it shows satisfaction and favorable product experience. This was short and correct.

Example	Prediction	Generated Output / Analysis
3	negative / long	Generated a positive explanation even though prediction was negative. This shows bias from positive-heavy data.
• Strengths: readable text, expected explanation style, good positive examples, and strong RAG perplexity improvement. • Weaknesses: repetition, occasional sentiment mismatch, and limited diversity. • Main causes: CPU training, small decoder subset, imbalanced data, and template-based reference explanations.		

9. Hyperparameter Notes and Limitations

- Small model sizes were selected because training was done on CPU.
- Embedding dimension 128 was enough for retrieval and kept computation manageable.
- Two layers and four heads balanced performance and runtime.
- Main limitations: small dataset subset, class imbalance, CPU-only training, 1200 decoder samples, and template-style references.
- Possible improvements: larger balanced dataset, more epochs, GPU training, better reference explanations, sampling/beam search, and approximate nearest neighbor retrieval for larger datasets.

10. Conclusion

- The full Transformer + RAG pipeline was implemented successfully from scratch.
- The encoder produced sentiment predictions, length predictions, and 128-dimensional embeddings.
- The retrieval module achieved 73.56% top-1 same sentiment rate.
- The decoder generated explanations using retrieved context and causal self-attention.
- The RAG system achieved 7.7282 perplexity compared with 60.8991 for the no-retrieval baseline.
- Despite dataset imbalance and limited CPU training, the system met the main assignment requirements and showed clear retrieval benefit.